

BACKEND ARCHITECTURE REFERENCE

AdGen-Agentic

Autonomous Multi-Agent Ad Generation System

Version: **1.0 — Implementation Reference**

Status: **Pre-implementation (FYP)**

Stack: **Python · LangGraph · FastAPI · PostgreSQL · Redis · Qdrant**

Purpose of this document

This is a complete implementation reference for the AdGen-Agentic backend. It covers the architectural mental model, directory structure, the Runtime-Agent separation pattern, the Event Bus, memory subsystem, tool gateway, database schema, and the LangGraph pipeline wiring. Read this before writing a single line of code.

1. Architectural Philosophy

1.1 What we learned from production systems

Before designing this system, we studied three production agentic platforms: OpenHands V1 (18 months of production lessons, ICLR 2025), Claude Code (reverse-engineered, ~50k LOC), and recent academic work on event-sourced agent memory. The following principles are directly borrowed from their hard-won lessons.

Principle	Why and where it comes from
Stateless agents	Agents are pure functions. They receive a compiled context, return an Action. No internal state. All mutable data lives in the Runtime's EventLog. (OpenHands V1)
Event-sourced state	All agent actions and tool results are immutable, append-only events. State is a projection over the log, not a mutable object. Enables crash recovery and deterministic replay. (OpenHands V1 + production research)
Runtime as provider	The Runtime owns memory, tools, and context compilation. Agents never call tools directly or manage their own memory. (OpenHands V1 SDK architecture)
Permission policy	Each agent is whitelisted to a specific set of tools. The Researcher cannot call ImageGen. The Producer cannot call VLMScore. Privilege isolation per agent. (Claude Code 6-layer security gate)
Context budget management	Before every agent turn, history is pruned to fit the token budget while preserving critical events. (Claude Code 92% threshold compressor)
Deterministic replay	If a campaign run crashes at step 7, load the EventLog, replay events 1-6, resume from step 7. No re-running completed work. (OpenHands V1 + event-sourcing research)

1.2 The three laws — tattoo these on the wall

Law 1 — Agents are stateless

An agent receives context, returns one Action. Nothing more. It does not remember previous turns itself, does not call tools itself, does not write to any database itself. The Runtime mediates everything.

Law 2 — The EventLog is the truth

There is exactly one authoritative record of what happened in a campaign run: the append-only EventLog stored in Postgres. Every other data structure (working memory, the LangGraph state object) is a projection derived from the EventLog. If they disagree, the EventLog wins.

Law 3 — The Runtime is the provider

Memory, tools, context compilation — the agent receives all of this pre-assembled from the Runtime. The agent's only job is to reason over what it is given. Swapping a tool, upgrading a memory backend, or changing the LLM provider never touches agent code.

2. System Layers Overview

The backend is composed of six layers. Each layer has a single responsibility and communicates with adjacent layers through typed contracts only.

Layer	Responsibility
Transport	FastAPI REST endpoints + WebSocket (live stream) + SSE fallback. Auth middleware (JWT). Rate limiter. The only public surface of the system.
Session manager	One CampaignSession per campaign run. Dispatched to Celery worker. Owns the WebSocket channel for that run. Manages heartbeat and timeout.
Agent Runtime	The core of the system. Owns EventBus, Memory subsystem, ToolGateway, ContextCompiler. Provides everything agents need. Never exposes internals to agents.
Agents	Four stateless agents: Researcher, Strategist, Producer, Auditor. Each is a class with a single step(context) -> Action method. No other public methods.
Tool executors	One file per external service: Firecrawl, LLM client (LiteLLM), Flux.1, Luma, ElevenLabs, asset store (R2/S3), VLM scorer. Called only by the Runtime's ToolGateway.
Persistence	Postgres (event_log, businesses, campaigns, assets, audit_logs). Redis (session state, Celery broker, SSE queue). Qdrant (BKO vector embeddings). R2/S3 (generated files).

3. Directory Structure

Every directory has a single owner. The runtime/ package is the most important — it must be implemented and tested before any agent code is written.

```

adgen/
├── runtime/                                # THE CORE. Everything agents depend on.
│   ├── session.py                        # CampaignSession – one per campaign run
│   ├── event_bus.py                      # Append-only EventLog + WebSocket pub/sub
│   ├── context_compiler.py               # Builds prompt for each agent turn
│   ├── tool_gateway.py                   # Typed registry + permission enforcement
│   ├── checkpointing.py                  # Persist + resume from EventLog
│   └── memory/
│       ├── working.py                    # Current campaign state (in-memory)
│       ├── episodic.py                   # Past campaign results (Postgres)
│       └── semantic.py                   # BKO vector search (Qdrant)
├── agents/                               # Stateless. step(context) → Action only.
│   ├── base.py                           # BaseAgent abstract class
│   ├── researcher.py
│   ├── strategist.py
│   ├── producer.py
│   └── auditor.py
├── actions/                              # Typed Pydantic action contracts
│   ├── web.py                            # WebScrapeAction
│   ├── llm.py                            # LLMCallAction, VLMScoreAction
│   ├── generation.py                     # ImageGenAction, VideoGenAction, VoiceAction
│   └── storage.py                         # StoreAssetAction
├── tools/                                # Executors. Called by Runtime only.
│   ├── firecrawl.py
│   ├── llm_client.py                     # LiteLLM wrapper (swap Claude ↔ GPT)
│   ├── flux.py
│   ├── luma.py
│   ├── elevenlabs.py
│   └── asset_store.py                    # R2/S3
├── pipeline/                             # LangGraph wiring only
│   ├── graph.py                          # Node registration + conditional edges
│   ├── edges.py                          # Routing logic (Auditor → retry or done)
│   └── checkpoints.py                     # LangGraph Postgres checkpointer
├── api/
│   ├── main.py                           # FastAPI app factory
│   ├── routes/
│   │   ├── businesses.py                 # POST /businesses, GET /businesses/{id}
│   │   ├── campaigns.py                 # POST /campaigns, GET /campaigns/{id}
│   │   └── stream.py                     # WS /campaigns/{id}/stream
│   └── middleware/
│       ├── auth.py
│       └── rate_limit.py
├── db/
│   ├── models.py                         # SQLAlchemy ORM models
│   ├── session.py                         # Engine + session factory
│   └── migrations/                       # Alembic
└── schemas/                             # Pydantic request/response contracts

```

```
| | business.py
| | campaign.py
| | event.py          # Action + Observation base types
| | asset.py
|
| tasks/
|   └─ campaign_task.py    # Celery task: runs CampaignSession.execute()
|
| prompts/              # All prompt templates. Never inside agent code.
|   └─ researcher.py
|   └─ strategist.py
|   └─ producer.py
|   └─ auditor.py
|
| config.py             # Env vars, API keys, feature flags
| docker-compose.yml    # postgres + redis + qdrant + app
| requirements.txt
| tests/
|   └─ test_runtime/
|   └─ test_agents/
|   └─ test_tools/
```

Rule

Prompts live in prompts/, never hardcoded inside agent files. Agents change rarely; prompts change constantly. Keep them separated from day one.

4. The Agent Runtime

The Runtime is not a framework — it is your code. It is the environment that agents live in. It owns four subsystems: the EventBus, the Memory layer, the ToolGateway, and the ContextCompiler. These are described below.

4.1 The Event Bus

Every action an agent takes and every tool result that comes back is an immutable Event appended to the log. Nothing is mutated. The EventLog is the authoritative record of the campaign run.

```
# runtime/event_bus.py

from pydantic import BaseModel, Field
from datetime import datetime
from typing import Literal, Any
import uuid

class Event(BaseModel):
    id: str = Field(default_factory=lambda: uuid.uuid4().hex)
    campaign_id: str
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    kind: Literal['action', 'observation', 'system']
    agent: str | None = None # which agent emitted this
    payload: dict # action or observation data

class EventBus:
    def __init__(self, campaign_id: str, db, ws_channel):
        self.campaign_id = campaign_id
        self._log: list[Event] = []
        self._db = db # Postgres session
        self._ws = ws_channel # WebSocket broadcaster

    def emit(self, event: Event) -> None:
        self._log.append(event) # 1. in-memory
        self._persist(event) # 2. Postgres (durable)
        self._ws.broadcast(event.dict()) # 3. live to frontend

    def history(self) -> list[Event]: # immutable view
        return list(self._log)

    def replay_from_db(self) -> None: # crash recovery
        events = (self._db.query(EventLog)
                  .filter_by(campaign_id=self.campaign_id)
                  .order_by(EventLog.timestamp).all())
        self._log = [Event(**e.payload) for e in events]
```

4.2 The Three Memory Layers

The memory subsystem has three distinct layers, each serving a different time horizon. The Runtime queries all three and the ContextCompiler assembles their outputs into the agent's prompt.

Layer	What it stores and when it is used
Working memory	In-memory Python dict. Current campaign only. Holds latest strategy_doc, asset_urls, audit scores. Wiped after the run completes. Zero latency — no DB calls.
Episodic memory	Postgres table. Past campaigns for this business. The ContextCompiler retrieves the 3 most recent campaigns with their top-performing hooks and alignment scores. Gives agents a 'what worked before' signal.
Semantic memory	Qdrant vector store. The Business Knowledge Object (BKO) is chunked, embedded, and stored here at onboarding. Retrieved by similarity query before every agent turn. This is how agents 'know' the business deeply without re-reading the whole BKO each time.

4.3 The Tool Gateway

The ToolGateway is the only way tools are called. Agents never import from tools/ directly. This enforces the permission policy and makes tool calls auditable.

```
# runtime/tool_gateway.py

PERMISSIONS = {
    'researcher': ['WebScrapeAction', 'LLMCallAction'],
    'strategist': ['LLMCallAction'],
    'producer': ['LLMCallAction', 'ImageGenAction',
                'VideoGenAction', 'VoiceAction', 'StoreAssetAction'],
    'auditor': ['VLMScoreAction', 'LLMCallAction'],
}

class ToolGateway:
    def permitted_tools(self, agent_name: str) -> list[str]:
        return PERMISSIONS.get(agent_name, [])

    def execute(self, action: BaseAction, agent_name: str) -> Observation:
        if action.__class__.__name__ not in self.permitted_tools(agent_name):
            raise PermissionError(f'{agent_name} cannot call {type(action).__name__}')
        executor = self._registry[type(action)]
        return executor.run(action)
```

4.4 The Context Compiler

Before every agent turn, the ContextCompiler assembles everything the agent needs into a single AgentContext object. It prunes history to fit the token budget, injects BKO chunks, and adds episodic memory snippets. The agent receives this and only this.

```
# runtime/context_compiler.py

class AgentContext:
    campaign_goal: str
    business_profile: dict # from semantic memory (BKO)
    strategy_doc: dict | None # None until Strategist has run
```



```
event_history:      list[Event]    # pruned to token budget
available_tools:    list[str]      # only permitted tools
memory_snippets:    list[str]      # from episodic memory
asset_critique:     str | None     # Auditor critique if re-rolling
token_budget:       int

class ContextCompiler:
    def __init__(self, token_budget: int = 32_000):
        self.budget = token_budget

    def build(self, agent, event_bus, memory, tool_gw) -> AgentContext:
        bko          = memory.semantic.retrieve(agent.query_terms)
        episodes      = memory.episodic.get_recent(n=3)
        history       = self._prune(event_bus.history()) # keep within budget
        tools         = tool_gw.permitted_tools(agent.name)
        critique      = self._last_critique(event_bus)   # if Auditor rejected
        return AgentContext(
            business_profile=bko, memory_snippets=episodes,
            event_history=history, available_tools=tools,
            asset_critique=critique, token_budget=self.budget
        )

    def _prune(self, events: list[Event]) -> list[Event]:
        # Always keep: system events, last Auditor critique
        # Trim: middle observations once over 80% of budget
        ...
```

5. The Agents

Each agent is a class with a single public method: `step(context: AgentContext) -> BaseAction`. Nothing else. No `__init__` state, no database access, no tool imports.

5.1 BaseAgent contract

```
# agents/base.py

from abc import ABC, abstractmethod
from runtime.session import AgentContext
from actions import BaseAction

class BaseAgent(ABC):
    name: str                # matches PERMISSIONS key
    query_terms: list[str]    # used by ContextCompiler for BKO retrieval

    @abstractmethod
    def step(self, ctx: AgentContext) -> BaseAction:
        """Receive context. Return exactly one Action. No side effects."""
        ...
```

5.2 Agent responsibilities

Agent	What it does and what it emits
Researcher	Receives campaign_goal + empty history. Emits WebScrapeAction (Firecrawl) then LLMCallAction to structure scraped content into the Business Knowledge Object (BKO). Writes BKO to working memory. If URL fails, falls back to free-text description in context.
Strategist	Receives BKO + campaign_goal. Uses Chain-of-Thought: first reasons about target customer psychology in the LLM call, then defines hook + platform-specific angles. Emits LLMCallAction with a structured JSON output schema. Output becomes strategy_doc in working memory.
Producer	Receives strategy_doc + asset_critique (if re-rolling). For each platform, emits ImageGenAction or VideoGenAction. Incorporates critique into the generation prompt if retry_count > 0. Emits StoreAssetAction for each file returned.
Auditor	Receives asset_urls + strategy_doc + BKO. Emits VLMScoreAction for each asset. Scores three dimensions: brand alignment (1-10), platform fit (1-10), hook strength (1-10). Emits LLMCallAction (two critic personas debate) to produce structured critique. Final score = weighted average.

Retry loop

If Auditor's weighted score < 7.0 AND retry_count < 3, LangGraph routes back to Producer. The critique from the Auditor's last observation is injected into the next Producer context via asset_critique. retry_count is incremented in the EventBus system event.

6. LangGraph Pipeline

LangGraph is used purely for graph wiring — node registration, conditional edges, and checkpointing. All business logic lives in the Runtime and agents. LangGraph's job is to call `CampaignSession.run_agent(agent)` in the right order.

6.1 Graph state

The LangGraph state is a thin projection of the EventBus. It carries the minimum needed for routing decisions. The source of truth remains the EventLog.

```
# pipeline/graph.py

from typing import TypedDict

class GraphState(TypedDict):
    campaign_id: str
    business_id: str
    campaign_goal: str
    platforms: list[str]
    # --- routing signals (updated by Runtime after each agent turn) ---
    audit_score: float          # latest Auditor weighted score
    retry_count: int            # incremented on each re-roll
    status: str                 #
    'researching'|'strategising'|...'done'|'failed'
```

6.2 Node wiring

```
from langgraph.graph import StateGraph, END
from pipeline.graph import GraphState
from runtime.session import CampaignSession
from agents import Researcher, Strategist, Producer, Auditor

def build_graph(session: CampaignSession):
    g = StateGraph(GraphState)

    # --- nodes: each calls session.run_agent() ---
    g.add_node('researcher', lambda s: session.run_agent(Researcher(), s))
    g.add_node('strategist', lambda s: session.run_agent(Strategist(), s))
    g.add_node('producer', lambda s: session.run_agent(Producer(), s))
    g.add_node('auditor', lambda s: session.run_agent(Auditor(), s))

    # --- linear edges ---
    g.set_entry_point('researcher')
    g.add_edge('researcher', 'strategist')
    g.add_edge('strategist', 'producer')
    g.add_edge('producer', 'auditor')

    # --- conditional edge: Auditor -> retry or done ---
    g.add_conditional_edges('auditor', route_after_audit,
        {'retry': 'producer', 'done': END})

    return g.compile(checkpointer=PostgresCheckpointer())
```

```
def route_after_audit(state: GraphState) -> str:
    if state['audit_score'] >= 7.0 or state['retry_count'] >= 3:
        return 'done'
    return 'retry'
```

6.3 Checkpointing

Use LangGraph's built-in Postgres checkpointer. Every node completion is checkpointed. On restart, the graph resumes from the last completed node — not from the beginning of the run.

```
# pipeline/checkpoints.py

from langgraph.checkpoint.postgres import PostgresSaver
from db.session import engine

def get_checkpointer():
    return PostgresSaver.from_conn_string(str(engine.url))
```

7. Database Schema

Five tables. Start exactly here — add columns later. The event_log table is the most important; everything else is derived from it.

businesses

```
CREATE TABLE businesses (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL,  
  name        TEXT NOT NULL,  
  website     TEXT,  
  bko         JSONB NOT NULL,    -- full Business Knowledge Object  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);  
CREATE INDEX idx_businesses_user ON businesses(user_id);
```

campaigns

```
CREATE TABLE campaigns (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,  
  goal        TEXT NOT NULL,  
  platforms   TEXT[],           -- e.g. ARRAY['linkedin','meta','tiktok']  
  strategy_doc JSONB,           -- populated after Strategist completes  
  status       TEXT DEFAULT 'pending',  
              -- pending | running | done | failed  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  completed_at TIMESTAMPTZ  
);
```

event_log (the authoritative record)

```
CREATE TABLE event_log (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  campaign_id UUID REFERENCES campaigns(id) ON DELETE CASCADE,  
  seq         BIGSERIAL,         -- deterministic ordering  
  kind        TEXT NOT NULL,     -- 'action' | 'observation' | 'system'  
  agent       TEXT,             -- which agent emitted this  
  payload     JSONB NOT NULL,    -- the full Action or Observation  
  created_at  TIMESTAMPTZ DEFAULT now()  
);  
CREATE INDEX idx_event_log_campaign ON event_log(campaign_id, seq);
```

assets

```
CREATE TABLE assets (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  campaign_id UUID REFERENCES campaigns(id) ON DELETE CASCADE,  
  platform    TEXT NOT NULL,    -- 'linkedin' | 'meta' | 'tiktok' | ...  
  format      TEXT NOT NULL,    -- '1:1' | '9:16' | '16:9'  
  asset_type  TEXT NOT NULL,    -- 'image' | 'video'
```

```
storage_url      TEXT,          -- R2/S3 URL after upload
alignment_score  FLOAT,         -- Auditor weighted score
retry_count      INT DEFAULT 0,
created_at       TIMESTAMPTZ DEFAULT now()
);
```

audit_logs

```
CREATE TABLE audit_logs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  asset_id      UUID REFERENCES assets(id) ON DELETE CASCADE,
  brand_score   FLOAT,          -- brand alignment dimension
  platform_score FLOAT,        -- platform fit dimension
  hook_score    FLOAT,          -- emotional hook strength dimension
  weighted_avg  FLOAT,          -- final score (0.4*brand + 0.35*hook +
0.25*platform)
  critique      TEXT,          -- structured verbal critique from Auditor
  iteration     INT,           -- which retry this is (0 = first attempt)
  created_at    TIMESTAMPTZ DEFAULT now()
);
```

BKO column (businesses.bko)

Store the full Business Knowledge Object as JSONB. Use `jsonb_path_query` for targeted retrieval. No migrations needed when you add fields to the BKO schema. The BKO is also chunked and embedded into Qdrant separately for semantic retrieval.

8. Async Execution Model

A campaign run takes 2-3 minutes. An HTTP request must return in milliseconds. The solution is a three-part async model: immediate response, background execution, and live streaming.

8.1 The three API endpoints

Endpoint	Purpose
POST /campaigns	Creates campaign row (status=pending). Enqueues Celery task. Returns {campaign_id} immediately. Never waits for the graph to run.
WS /campaigns/{id}/stream	WebSocket connection. The Celery worker writes events to a Redis channel. This endpoint relays them to the client in real time. Client sees agent reasoning as it happens.
GET /campaigns/{id}	Returns final campaign data once status=done. Assets, scores, strategy doc. Called by frontend after the WebSocket closes.
GET /campaigns/{id}/events	Returns full EventLog for a completed run. Used for debugging, replay, and the FYP evaluation audit trail.

8.2 Celery task

```
# tasks/campaign_task.py

from celery import Celery
from runtime.session import CampaignSession
from pipeline.graph import build_graph

app = Celery('adgen', broker='redis://localhost:6379/0')

@app.task(bind=True, max_retries=2)
def run_campaign(self, campaign_id: str):
    try:
        session = CampaignSession.load(campaign_id) # from DB
        graph = build_graph(session)
        graph.invoke(session.initial_state(),
                     config={'configurable': {'thread_id': campaign_id}})
    except Exception as exc:
        # mark campaign as failed, emit system event
        session.fail(str(exc))
        raise self.retry(exc=exc, countdown=30)
```

8.3 Crash recovery flow

1. Celery task crashes mid-run (e.g. at Producer step).
2. On retry, CampaignSession.load() calls event_bus.replay_from_db().
3. LangGraph Postgres checkpoint resumes from last completed node.

4. Producer re-runs (only Producer, not Researcher or Strategist).
5. No token waste, no duplicate work.

9. Business Knowledge Object (BKO)

The BKO is the foundation of everything. It is built once at onboarding and consumed by every campaign. Design it carefully — every field maps directly to a prompt variable used by the Strategist and Producer agents.

9.1 BKO schema

```
{
  "meta": {
    "name": "Acme SaaS",
    "website": "https://acme.io",
    "industry": "SaaS",
    "sub_category": "productivity / async teams",
    "business_model": "B2B subscription",
    "last_updated": "2025-06-01"
  },
  "identity": {
    "one_liner": "AI meeting summaries that cut standup time by 40%",
    "tone_of_voice": ["confident", "no-fluff", "data-driven"],
    "brand_colors": ["#1A1A2E", "#E94560"],
    "avoid": ["corporate jargon", "exclamation marks"]
  },
  "audience": {
    "primary": "SaaS founders, remote team leads, age 28-45",
    "pain_points": ["Too many unproductive meetings", "Context lost between async team members"],
    "desired_outcomes": ["Reclaim 5+ hours per week", "Team aligned without live calls"]
  },
  "value_props": [
    { "claim": "40% reduction in meeting time", "proof": "200 user study" },
    { "claim": "AI summaries in 30 seconds", "proof": "product feature" }
  ],
  "competitors": ["Fireflies.ai", "Otter.ai"],
  "differentiators": ["Fastest summary", "Async-first by design"],
  "platform_notes": {
    "linkedin": "ROI data, professional tone, target decision-makers",
    "meta": "Emotional pain/relief arc, visual product demo",
    "tiktok": "Fast-paced, founder story angle"
  },
  "campaign_history": [
    {
      "campaign_id": "uuid",
      "goal": "Free trial signups",
      "top_performing_hook": "Your team is in 6 meetings a day...",
      "alignment_score": 9.1
    }
  ]
}
```

9.2 Three onboarding paths

All three paths produce the same BKO structure. The Researcher agent handles the extraction and structuring for paths 1 and 2. Path 3 is a direct form-to-BKO mapping.

Path	How it works
URL input	Firecrawl scrapes homepage, pricing, and about pages. LLM structures the content into BKO fields. Gaps are flagged to the user for confirmation. 60 seconds total.
Free description	User writes a paragraph. LLM extracts fields. Missing fields trigger follow-up questions: 'You mentioned restaurants — what is their main pain point?' Best for pre-launch businesses.
Onboarding form	5 structured questions mapped directly to BKO fields. Highest quality input. Recommended for first-time users. Guaranteed completeness, no inference needed.

10. Implementation Order

Do not skip steps. Each phase depends on the previous. The Runtime must be fully working before any agent is written. Agents must work before the LangGraph pipeline is wired.

Phase 1 — Foundation (Week 1-2)

Goal: a running server with a working database, Redis, and a stub campaign endpoint.

6. Set up docker-compose.yml with Postgres, Redis, Qdrant.
7. Create all 5 DB tables with Alembic migrations.
8. Build FastAPI skeleton: POST /campaigns returns {campaign_id} immediately.
9. Wire Celery: task enqueues and runs, logs 'hello world'.
10. Write EventBus class with emit() and history() — no DB yet, in-memory only.

Checkpoint

You should be able to POST /campaigns and see a Celery task log. Nothing AI yet. That's correct.

Phase 2 — Runtime core (Week 3-4)

Goal: the Runtime works end to end with stub agents and fake tool responses.

11. Implement ToolGateway with permission enforcement.
12. Implement ContextCompiler with stub memory (returns empty lists).
13. Write BaseAgent and four stub agents that return hardcoded Actions.
14. Implement CampaignSession.run_agent() loop.
15. Add EventBus Postgres persistence and WebSocket broadcast.

Checkpoint

Run a full pipeline with stub agents. Every action and observation should appear in event_log. Replay from DB should reconstruct identical state.

Phase 3 — Real agents (Week 5-8)

Replace stub agents one by one. Test each in isolation before wiring.

16. Researcher: wire Firecrawl + LLM, write BKO to working memory.
17. Semantic memory: embed BKO chunks into Qdrant, wire ContextCompiler retrieval.
18. Strategist: CoT prompt, structured JSON output, write strategy_doc.
19. Producer: Flux.1 integration for static images, StoreAssetAction to R2.
20. Auditor: GPT-4o-vision scoring, two-persona critique, conditional retry edge.

Phase 4 — Video + voice (Week 9-10)

Add Luma Dream Machine and ElevenLabs to the Producer agent. These are additive — do not change the Producer's `step()` signature.

Phase 5 — Episodic memory + BKO enrichment (Week 11)

Wire episodic memory retrieval into `ContextCompiler`. After each completed campaign, write the top hook and alignment score back to the BKO's `campaign_history` array.

Phase 6 — Evaluation + hardening (Week 12-14)

21. Instrument all tool calls with latency logging.
22. Add retry logic and timeouts to each tool executor.
23. Run 10 test businesses end-to-end. Collect latency, alignment scores.
24. Run A/B user study: AdGen output vs one-prompt baseline.
25. Write evaluation results to dissertation.

11. Technology Quick Reference

Technology	Usage notes
FastAPI	REST + WebSocket server. Use async def everywhere. Pydantic v2 for request/response validation.
Celery + Redis	Celery for async task queue. Redis as broker and result backend. One worker process per campaign run.
LangGraph	Graph orchestration only. Nodes, conditional edges, Postgres checkpointer. All logic in Runtime.
LiteLLM	LLM client wrapper. Swap Claude <-> GPT-4o in one config line. Handles retries and rate limits.
SQLAlchemy 2	ORM with async session support. Use mapped_column() and Mapped[] for type safety.
Alembic	DB migrations. Run alembic revision --autogenerate for every schema change.
Qdrant	Vector store for BKO semantic search. Run locally via Docker for dev; Qdrant Cloud for prod.
Firecrawl	Web scraping API. Handles JS-rendered pages, rate limiting, and content extraction automatically.
Flux.1	Image generation via Replicate API. Use flux-1.1-pro model. Specify exact pixel dimensions.
Luma Dream Mach.	Video generation API. Async job model: submit prompt, poll for completion, download MP4.
ElevenLabs	Voiceover API. Use streaming for long narrations. Cache audio files in R2.
Cloudflare R2	Asset storage. S3-compatible API. Free egress. Use boto3 with custom endpoint_url.
Pytest	Write unit tests for every Runtime class and tool executor before wiring into agents.
Docker Compose	Local dev: postgres + redis + qdrant + app in one command. Match prod image in dev.

12. Rules to Never Break

- Agents never import from tools/. Ever.
- Agents never write to the database directly.
- Agents never call each other. Use AgentDelegateAction via the Runtime if sub-delegation is needed.
- Prompts live in prompts/, never hardcoded in agent files.

- The EventLog is append-only. Never update or delete event rows.
- Every tool call goes through ToolGateway. No exceptions for 'quick' calls.
- GraphState carries only routing signals. All real state is in the EventLog.
- Test the Runtime in isolation before writing any agent. It is the hardest part.
- Never change the BKO schema without a migration plan. The Qdrant embeddings must be re-indexed if BKO fields change.
- Every external API call (Firecrawl, Flux, Luma, ElevenLabs) has a timeout and a fallback. Do not let a slow API hang the entire campaign run.